

G1 Digital-Twin Payload Experiment

Methods, spill model, and governance algorithm — supporting material for Section VII.H

Simulation campaign 2026-07-04 · Unitree G1 · Gazebo (ROS 2 Humble) · 83 runs, 11 arms

1. Digital twin: Gazebo replica of the laboratory

The physical robot is a Unitree G1 “Air” humanoid, a consumer unit whose only external interface is a single WebRTC session through the vendor app. The production navigation stack drives it by tapping that session over USB. For the twin we reuse **the exact same stack, unmodified**, and substitute only its interface object, so any behaviour observed in simulation is produced by the identical code deployed on hardware.

The simulator runs inside a Docker container (ROS 2 Humble Desktop with Gazebo Classic, accessible over noVNC). The world, **lab.world**, is generated programmatically from the robot's own SLAM maps of the real laboratory: wall occupancy cells become 2.2 m box obstacles in the robot's coordinate frame, so the twin's geometry, waypoints (A, B, C), and the 0.8 m doorway coincide with the real deployment. The doorway was re-carved to match the measured physical opening after the collision model was corrected (see §2).

1.1 Robot model and physics

The robot is spawned from a URDF whose collision geometry is a single box matching the G1's real envelope (0.44 m shoulder width × 1.10 m tall, capped below the simulated LiDAR at $z \approx 1.20$ m so the sensor does not see itself). A high mass (60 kg) and large diagonal inertia keep the base from tipping under the velocity-controlled gait; ground friction is set to zero so the planar-move plugin drives it as a velocity-controlled puck while wall contacts remain solid. Locomotion is commanded as gait velocity through **gazebo_ros_planar_move** (`/cmd_vel` → base motion, `/odom` published back`); a 360-ray LiDAR at 10 Hz publishes `/scan``.

1.2 The adapter: same stack, simulated interface

A thin adapter (**SimCDP**) replaces the WebRTC/WebView interface object with one that resolves the stack's own calls against ROS 2 over rosbridge (WebSocket, port 8765):

- **Velocity commands** `{lx, ly, rx, ry}` → `/cmd_vel``, using the physical robot's measured stick-to-velocity mapping (deadzone subtracted, not clipped; $lx > 0$ = right; $ly = 0.40 \rightarrow 0.30$ m/s), so simulated and real dynamics coincide.
- **Pose** $\leftarrow$ `/odom`` (in simulation the map and odometry frames coincide, so no relocalisation jitter).
- **LiDAR** $\leftarrow$ `/scan``, the 360-ray sweep projected into a flat world-frame point cloud identical in shape to what the real WebView decoder produces.

Camera perception is disabled in the twin (no visual channel); the LiDAR-driven planner and the governance layer are exercised in full. Data runs are always headless; the CPU renderer would otherwise steal cycles from physics.

2. Spill model (payload hazard)

Spilling is modelled physically, not as an arbitrary speed threshold. The water surface at the cup rim is a **damped second-order oscillator** — the first sloshing mode of an open cylinder — excited by the horizontal acceleration of the cup:

$$\omega_0 = \sqrt{g \cdot k_1 \cdot \tanh(k_1 \cdot h)}, \quad k_1 = 1.841 / R \quad (\approx 21 \text{ rad/s for } R = 4 \text{ cm, } h = 8 \text{ cm})$$

$$\eta_{\text{target}} = -R \cdot a_{\text{cup}} / g \quad (\text{steady tilt of the surface})$$

$$\ddot{\eta} = -2 \cdot \zeta \cdot \omega_0 \cdot \dot{\eta} - \omega_0^2 \cdot (\eta - \eta_{\text{target}}) \quad \text{per body axis, } \zeta = 0.06$$

The cup is held in the hand ~0.25 m ahead of the base, so $\ddot{a}_{\text{cup}} = \ddot{a}_{\text{base}} + \alpha \times r + \omega \times (\omega \times r)$: turning fast adds a lever term. Bipedal gait injects an acceleration noise that grows with speed ($\approx v^2$, a 1.4 Hz component plus broadband) — **walking fast agitates the cup by itself**, independent of any single manoeuvre. A sharp deceleration while forward motion is still commanded (an external impact, not a commanded stop) adds a direct impulse. Commanded acceleration is low-pass filtered ($\tau = 0.25 \text{ s}$) because the real G1 ramps velocity smoothly whereas the planar-move plugin steps it instantaneously.

A spill is drawn as a **non-homogeneous Poisson process** whose rate rises with how far the surface exceeds the freeboard (the 12 mm of clearance below the rim):

$$\lambda(t) = \text{LID} \cdot \lambda_0 \cdot \max(0, |\eta|/\text{freeboard} - 1)^2, \quad \lambda_0 = 40 \text{ s}^{-1}, \quad \text{LID} = 1.0 \text{ (open)} / 0.25 \text{ (sealed)}$$

Each spill loses a little water (freeboard grows) and dissipates energy. The model is **observational only** — it never alters the physics or the control. It is fed the odometry stream at ~30 Hz and emits spill events with timestamps into the per-run dataset; continuous diagnostics (expected spill count = $\int \lambda \, dt$, peak $|\eta|/\text{freeboard}$, fraction of time in the >0.7 risk band) are recorded alongside. Calibration against logged trajectories gives ≈ 2 spills per 80 s of sustained 0.30 m/s cruising and ≈ 0 at the governed 0.18 m/s ceiling, consistent with the jerk-dominated design of the thesis 2-D study.

For the physical robot the same event channel accepts a **manual ground-truth mark**: a human presses a key (or publishes an empty ROS message) each time water is seen to spill, and the event is timestamped into the identical dataset field, so simulated and real runs are scored the same way.

3. Governance algorithm (DCE runtime)

The governance layer is the configuration-first meta-reasoner (Meta-Reasoner 2.0) driven through a bridge that packages the stack's per-tick telemetry as shared-experience readings at 2 Hz. Each tick the reasoner evaluates the candidate analogies (behaviour models, e.g. Efficient_Nav / Cautious_Nav) against their QoE boundaries and returns one of KEEP / SWITCH / FALLBACK / HELP / INSUFFICIENT plus the active analogy; the bridge maps the decision to a forward-speed ceiling applied to the controller (Efficient = uncapped, Cautious = 0.28, FALLBACK = 0.24, HELP = 0).

3.1 Shared-experience mapping

Meta-parameter	Source (per-tick telemetry)	Role
safety	clearance = c_0 / 1.5 m (LiDAR forward free space)	QoE region + hard veto
progression	goal-approach rate / 0.30 m/s (2 s window)	required-meta threshold
mobility	achieved speed / commanded speed (1.2 s)	resistance channel, hard veto
fragility	payload state: 1.0 intact, drops per spill	hard veto (payload task)
reliability / uncertainty	sensing monitor + few-shot historical margin	DST belief interval

Readings are median-smoothed over ~ 2.5 s; each reading's DST uncertainty adds the historical dispersion of its own metric (few-shot margin), so a value oscillating across a QoE boundary widens its belief/plausibility interval and cannot be certified on a single lucky sample. Switches are hysteretic (confirmed after 3 consecutive winners, released more slowly). An experience-escalation layer aborts the run when HELP is sustained (≥ 8 s) or a 75 s window shows $\geq 60\%$ FALLBACK/HELP with < 0.4 m net progress — the runtime's specified “request intervention” semantics.

3.2 Two-layer trust (thesis Ch. 5 realised on the G1)

Intra-run (fragility channel). Each spill — from the sim model or the human mark — drives the fragility meta-parameter: one spill lowers it to caution, two to FALLBACK, three or more into the dangerous region where the hard veto forces HELP; it recovers on a ~ 25 s time constant if no further spill occurs. A spill is treated as a certain event, not a noisy sensor reading.

Cross-run (Layer 2). A per-analogy Dempster–Shafer belief over {match, mismatch, $\emptyset$ } is persisted between runs. At each run's end the outcome is fused by Dempster's rule — a spill under analogy A adds mass to A's mismatch; a clean run adds match to the governing analogy. The plausibility $PI = m_{\text{match}} + m_{\emptyset}$ then gates deployment: below 0.70 the analogy's speed ceiling is blended toward the conservative policy (the thesis policy blend), below 0.45 the analogy is not deployed and a discredited initial prior is replaced at start-up. In a controlled test this reproduces the thesis M2+Wrong $\rightarrow$ DST recovery: $PI(\text{Efficient})$ degrades $1.00 \rightarrow 0.75 \rightarrow 0.53 \rightarrow 0.36$ over three spill runs and the fourth run starts pre-corrected. The reasoner itself is untouched; both layers live in the bridge.

4. Campaign protocol

Each run resets Gazebo to a fresh world with the robot at waypoint A, then executes an autonomous A $\rightarrow$ B traverse headless with a timeout (a SIGINT saves the dataset exactly as a manual stop would). Governance arms are alternated so drift in machine load is shared across conditions. Arms are

selected by environment configuration; every run is tagged `env=sim` with a per-arm identifier, and the aggregation script reports mean $\pm$ 95% CI ($1.96 \cdot s/\sqrt{n}$, matching the thesis protocol), reached/abort rates, and — for the cross-run arms — the per-run recovery sequence with the final Layer-2 belief state. Datasets, the aggregation script, and the exact configurations are versioned with the code.

5. Results

Governance arms (N = 8 runs each; sealed-lid arms N = 6; mean $\pm$ 95% CI). Risk is the fraction of run time with the liquid surface above 70% of freeboard; HELP is a governed abort following a sustained empty deployable set.

Arm / governance	Reached	Time (s)	Spills/run	Risk (%)
M0 baseline — none (0.30 m/s)	8/8	83.4 $\pm$ 5.2	0.50 $\pm$ 0.37	8.1 $\pm$ 1.4
M1 fixed-conservative — none (0.18 cap)	8/8	110.8 $\pm$ 2.4	0.00	0.8 $\pm$ 0.2
Single-candidate governed	5/8 (3 HELP)	106.2 $\pm$ 5.1	0.12 $\pm$ 0.24	0.5 $\pm$ 0.2
M2 payload (proposed)	8/8	107.9 $\pm$ 4.2	0.00	0.4 $\pm$ 0.2
M2 + task-blind prior	8/8	108.2 $\pm$ 3.5	0.00	0.7 $\pm$ 0.2
M0, sealed lid	6/6	88.1 $\pm$ 4.9	0.17 $\pm$ 0.33	10.6 $\pm$ 2.9
M2, sealed lid (twin-calibrated)	5/6 (1 HELP)	102.5 $\pm$ 2.2	0.17 $\pm$ 0.33	0.7 $\pm$ 0.5

The five findings and the physical-robot protocol are stated in Section VII.H of the manuscript; this document supplies the methodological detail behind them.